

Minecraft Intrusion Detection System

Oliver Wright

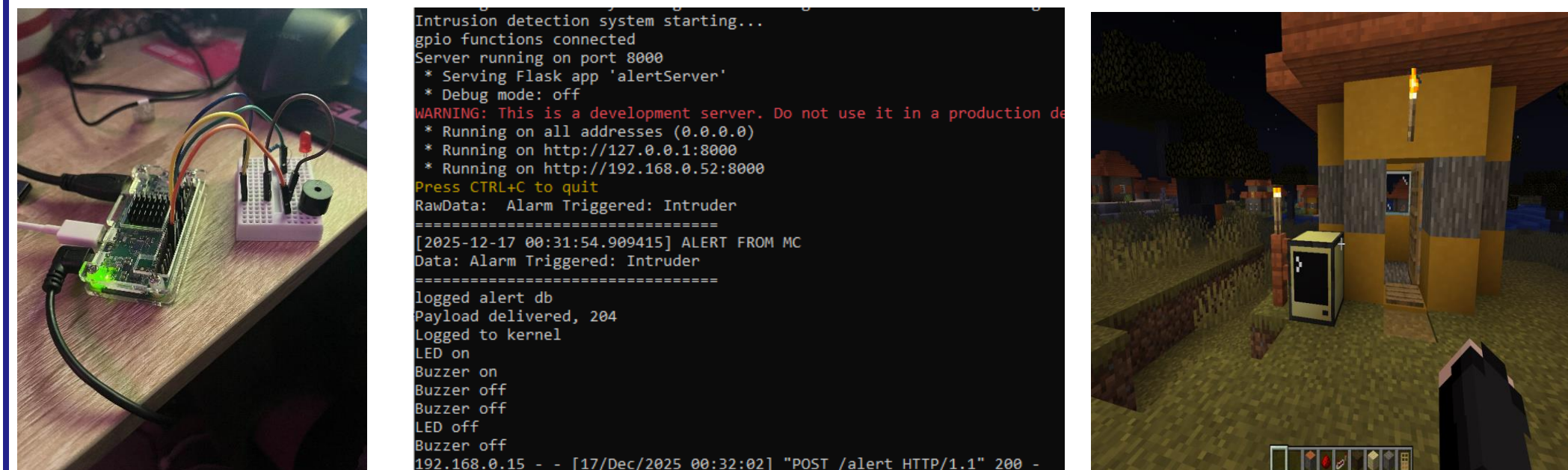
CMP408

Introduction

Due to the quickly developing Internet of Things (IoT) landscape, production often fails to meet demand. Due to this aspects of the development cycle are sometimes rushed, resulting in the proper security precautions not being taken for the device in question. Another common problem is the hardware constraints on IoT devices. "As the number of connected IoT devices continually increase, security issues are exponentially multiplied [...]. Moreover, traditional security mechanisms cannot be directly implemented to IoT technologies due to their designed system i.e. limited power as well as these large number of connected devices raise heterogeneity and scalability issues", (Hanan, Shafiq, & Irfan, 2019).

This project makes use of a Raspberry Pi (RPI) as the platform for the IoT device and holds most of the software files and logic. Attached to the RPI via breadboard is a circuit containing an LED and a Buzzer. These components' purpose is to alert the user of when an intrusion has been detected by the system. Upon receiving an alert, notifications are sent to the user through discord, creating a higher chance of successfully notifying the user. These alerts are also stored as logs in a database, saving each one with an ID and a timestamp. Lastly, the system has a web application to go alongside its current features. This web application functions as a security dashboard, allowing the user to view the alert logs in the database. The dashboard contains a refresh button in case an alert comes through while on the webpage.

Methodology



- The project is an intrusion detection system (IDS) involving secure IoT, cloud, and software development.
- A simple HTTP flask server was run on the RPi using a python script "alertServer.py". This script waits to receive an alert before continuing to trigger any of the other IDS features. Next, an instance of Minecraft with the mod CC:Tweaked was launched. Here a computer was placed and a Lua script to POST HTTP requests was setup. An API key was included in the requests which is checked by the verifyKey function in alertServer. This prevents false requests being sent from other applications. The RPi also had hardware components in the form of an LED and a buzzer. These are attached using a breadboard and configured in the config file. The flipLED and flipBuzz functions are called after receiving an alert, and they cause the LED and buzzer to activate to alert the user. Threading is used here to allow them to work side by side without interrupting each other.
- When an alert is received, a timestamp is taken and it is assigned an ID. This is then sent as an item to a database with the fields (ID, message, timestamp). DynamoDB was used to hold the database due to its scalability feature. It receives alerts exclusively from the RPi as the 3 different API keys are required and well hidden on the device. Amongst the python files is a .env file. This file holds environment variables and is not uploaded to the GitHub repository. All this project's sensitive variables are stored here and called from the config file using the "os" and "dotenv" libraries.
- Next the security dashboard web application was developed and hosted using AWS Amplify. This allows users to view their previous intrusion alerts as well as receive them in real-time. It was some basic HTML and CSS made the web application look respectable. The JavaScript receives the databases alert logs inside a JSON dump and separates them accordingly. Next each item creates a new card instance and is displayed on the page. Lambda and API gateway were used to connect the database to the web application. A Lambda function acquires the database logs and sorts them into a list to display the newest alerts first.
- Alongside the IDS an LKM was developed to monitor the RPi's GPIO pins. The module increments and alert counter by 1 every time it detects power through the GPIO 3V pin attached to the LED. This is launched as a service and runs alongside the rest of the program.

Project Highlights

- An API key was generated using a python script and placed into the ".env" file as an environment variable. If POST requests to the HTTP server did not include this API key, then they were deemed invalid.
- A sanitize message function was implemented to remove certain characters from the message before sending it to the database. This prevents the potential for SQL injection.
- An environment file was used to store sensitive variables to prevent them from being exposed to unwanted persons.

Future Work

- More methods of notifying the user of intrusions could be implemented. This could be through email, text, or via an app.
- The security dashboard web application could be further developed to provide more information such as flags on whether each intrusion has been dealt with.

References

Hanan, A., Shafiq, U. R., & Irfan, U. (2019, July). Security in Internet of Things: Issues, Challenges and Solutions. Retrieved from ResearchGate:
https://www.researchgate.net/publication/326579980_Security_in_Internet_of_Things_Issues_Challenges_and_Solutions